

AI hudební videoklip z prohlížeče: technické možnosti a realistický stack pro vibe-codera (2026)

TL;DR

- **Nejschůdnější cesta:** Hybridní PWA, která dělá v prohlížeči jen to, co prohlížeč umí dobře — audio analýzu (Web Audio API), real-time audio-reaktivní vizuál (Three.js `WebGPURenderer` + TSL, nebo Butterchurn jako MVP) a finální skládání/export (WebCodecs `VideoEncoder` + `mp4-muxer`, případně `ffmpeg.wasm` pro audio mux). Pro „opravdové AI obrazy“ volá externí API (Fal.ai s Flux 2 a Wan/Pika/Luma image-to-video). Plné generování diffusion videa v prohlížeči je v roce 2026 stále nepraktické.
- **Pro koho je která cesta:** Čistě algoritmický audio-reaktivní vizuál (shadery, particles, Butterchurn) = zdarma, deterministický, real-time export, ale „demoscene/VJ“ vzhled. Hybrid s AI keyframy a image-to-video API = vypadá jako moderní klip z Neural Frames/Kaiber, ale stojí cca \$8-40 za 3minutový klip a renderuje minuty. Plně AI v prohlížeči (SD přes WebGPU) = funguje pro testovací keyframy, ne pro plnohodnotný klip.
- **Stack, který doporučuji:** React 19 + Vite + TypeScript + Tailwind + `vite-plugin-pwa` + Three.js (`WebGPURenderer` s automatickým fallbackem na WebGL2) + TSL pro shadery + Meyda (beat/feature detection) + WebCodecs `VideoEncoder` + `mp4-muxer` + Fal.ai SDK (volitelně) + `ffmpeg.wasm` pro finální audio mux. Tohle je realistický „vibe-coding“ terén — všechno má dokumentaci, kterou Claude/Gemini dobře znají.

Key Findings

1) Stav WebGPU k 2026 — můžeš na něj spolehlivě stavět, ale s fallbackem

WebGPU je od **25. listopadu 2025** default ve všech hlavních prohlížečích: Chrome/Edge 113+, Safari 26 (macOS Tahoe 26, iOS/iPadOS 26, visionOS 26), Firefox 141+ na Windows, Firefox 145+ na macOS ARM64. Globální pokrytí podle buildmvpfast.com (Nov 2025) skočilo na cca **82,7 %** v okamžiku, kdy všechny čtyři engine shipnuly WebGPU defaultně (starší 70% číslo z byteiota.com je z října 2024). Mobilní fragmentace zůstává: Firefox Android dosud za flagem, Mozilla plánuje Linux/Android v průběhu 2026. **Vždy nasazuj progressive enhancement s WebGL2 fallbackem** — Three.js `WebGPURenderer` to dělá automaticky (`forceWebGL: false`).

`web.dev + 4`

2) Audio analýza — Web Audio API stačí

`AudioContext` → `MediaElementAudioSource` → `AnalyserNode` (`fftSize` 2048) → `destination`. `getByteFrequencyData()` ti dá frekvenční bins, `getByteTimeDomainData()` waveform. Pro beat detection použij **spectral flux** (jednoduchá implementace) nebo přímo knihovnu **Meyda** (kterou používá Hydra), případně **essentia.js** pro pokročilou hudební analýzu (BPM, key, onsets, sekce). Pro hudební klip jsou rozumné hodnoty: `fftSize` 2048-4096, `smoothingTimeConstant` 0.8 pro klidnější vizuál, 0.3-0.5 když chceš reaktivní transienty. `GitHub`

Suno licenční rovina: Suno Free tier = non-commercial only. Pro YouTube monetizaci (Partner Program) potřebuješ Suno Pro nebo Premier plán. Pro srovnání: Udio po dohodě s Universal Music Group oznámené 29. října 2025 vypnulo stahování okamžitě pro **všechny uživatele včetně free-tier** (krátké 48hodinové okno 3. listopadu po vlně kritiky; Warner Music Group settlement následoval 19. listopadu 2025). Spotify nově vyžaduje AI disclosure přes DDEX credits distributora. [Digital Music News](#) [WebProNews](#)

3) Generativní (algoritmický) vizuál vs. AI vizuál — dvě úplně různé tradice

Aspekt	Generativní/algoritmický (shadery, particles, fluid)	AI-generovaný (diffusion, Wan, Flux+i2v)
Real-time	Ano, 60 FPS na slušném HW	Ne (sekundy až minuty na klip)
Cena	Zdarma	\$0.04-\$0.42 / sec videa via API
Vzhled	„VJ"/„demoscene"/abstraktní	Fotorealismus, kinematografie, narativ
Reaktivita na audio	Přesná, frame-by-frame	Pouze přes parametry (motion strength, prompt blending), ne přesně
Determinismus	Plně reprodukovatelné	Stochastické, drahé na iteraci
Vhodné pro	Spotify Canvas, abstraktní klipy, DJ sety	„Music video" s postavami, scénami

Pro vibe-codera doporučuju začít čistě algoritmicky a AI přidat až jako volitelnou vrstvu. Kvalitní audio-reaktivní shader (Codrops tutoriál „Coding a 3D Audio Visualizer with Three.js, GSAP & Web Audio API", červen 2025) vypadá v roce 2026 stále profesionálně a dá ti 100% kontrolu nad výsledkem.

4) Three.js + TSL je sweet spot pro WebGPU shadery

Three.js Shading Language (TSL) je JS-like DSL, který se automaticky kompiluje na WGSL (WebGPU) nebo GLSL (WebGL2 fallback). Píšeš shadery v JavaScriptu/TypeScriptu, takže Claude/Gemini ti je umí navrhnout výrazně líp než syrový WGSL. Migrace z `WebGLRenderer` na `WebGPURenderer` je často **jedno-řádková změna** (`import * as THREE from 'three/webgpu', new THREE.WebGPURenderer()`). Doporučuju R171+ z `three/webgpu` namespace. Codrops, ICS Media a Maxime Heckel (článek „Field Guide to TSL and WebGPU") mají publikované produkční tutoriály. Compute shadery ve WebGPU řeší particle systémy se 100k+ částicemi nebo fluid simulace, které WebGL2 nezvládne (WebGL nemá compute shadery vůbec). [byteiota](#)

Pro React 19 + React Three Fiber existuje async `gl` prop, který umí inicializovat `WebGPURenderer` (od R3F v9). Pokud nechceš R3F, čistý Three.js stačí.

5) AI obraz v prohlížeči — funkční, ale ne pro plné klipy

- **Stable Diffusion 1.5 v prohlížeči** přes WebGPU funguje: `mlc-ai/web-stable-diffusion`, `diffusers.js`, `Zhare-AI/sd-1-5-webgpu`. Praktická limita: ~8 GB RAM/VRAM, generace 512×512 trvá desítky sekund až minuty.
- **ONNX Runtime Web + WebGPU EP** spustí **Stable Diffusion Turbo** v sekundách. Microsoft Open Source blog (29. února 2024, „ONNX Runtime Web unleashes generative AI in the browser using WebGPU“) uvádí doslova: „*it is now possible to execute Stable Diffusion Turbo directly in the browser, achieving results within one second on an RTX 4090.*“ (Benchmark platí pro SD Turbo na RTX 4090; SDXL Turbo v tom postu kvantifikován není.) (`Microsoft Open Source`)
- **Žádný WebGPU port SVD (Stable Video Diffusion)** v roce 2026 veřejně neexistuje. Temporální vrstvy SVD se v aktuálním WebGPU compilation chain neimplementují.
- **AnimateDiff/Deforum v prohlížeči** = neexistuje. Vyžaduje plný SD pipeline + motion module (~1,7 GB), není MLC-zkompilované.

Praktický závěr: AI generování dále server-side přes API. WebGPU/ONNX Runtime Web v prohlížeči použij na lehčí věci — **RIFE frame interpolation** (`yuvraj108c/rife-onnx`, ~21,5 MB), **Real-ESRGAN upscaling**, embedding/encoder modely. I tady ale počítej s vlastní integrační prací — žádné public WebGPU demo RIFE inferenc-loopu jsem nenašel. (`Hugging Face`)

6) Externí API pro AI obraz/video — konkrétní ceny (Fal.ai, květen 2026)

Pro hybridní pipeline jsou nejzajímavější tyto endpointy:

Keyframes (still images):

- `fal-ai/flux/dev` — \$0.025/obrázek
- `fal-ai/flux-2-pro` — \$0.03 za první megapixel (1024×1024 = \$0.03; 1920×1080 = \$0.045)

Image-to-video (animace mezi keyframy nebo z jednoho keyframu):

- `fal-ai/wan/v2.2-a14b/image-to-video` — **\$0.04/sec (480p)**, \$0.06/sec (580p), \$0.08/sec (720p) při 16 FPS — nejlepší poměr cena/kvalita (`fal.ai`)
- `fal-ai/wan-flf2v` — **\$0.20–\$0.40 za 5s video** (480p/720p), explicitně „first-last-frame“ mód — ideální pro keyframe-to-keyframe synchronní s hudbou (`fal.ai`)
- `fal-ai/pika/v2.2/pikaframes` — **\$0.04/sec (720p)**, \$0.06/sec (1080p), podporuje až **5 keyframů s vlastním promptem na každý segment** — nejnaturálnější fit pro music video se sekcemi (`fal.ai`) (`fal.ai`)
- `fal-ai/luma-dream-machine/ray-2/image-to-video` — \$0.10/sec (Ray 2, \$0.50 za 5s 720p) nebo \$0.04/sec (Ray 2 Flash); podporuje `image_url` + `end_image_url` (dual keyframe) (`fal.ai`) (`fal.ai`)
- `fal-ai/kling-video/v2.6/pro/image-to-video` — \$0.07/sec bez audia, \$0.14/sec s audiem

- `fal-ai/veo3.1/image-to-video` — \$0.20-\$0.40/sec, native audio (Google Veo); Fast varianta \$0.10-\$0.15/sec

Frame interpolation (smoothing po stitching):

- `fal-ai/rife/video` — **\$0.0013 / compute-sec** — prakticky zdarma, RIFE upsample (např. 16 → 48 FPS)

Hrubý odhad ceny 3min klipu:

- 16 keyframů Flux Dev: \$0.40
- Animace přes Wan 2.2 (480p, 180 s): ~\$7.20
- RIFE smoothing: ~\$0.10
- **Celkem ~\$8 za render** + iterace; varianta s Pikaframes / Wan FLF2V podobně, varianta s Luma Ray 2 nebo Kling 4K spíše \$25-40.

Důležité: AnimateDiff prompt-travel přes Replicate (`replicate.com/zsxkib/animatediff-prompt-travel`) doslova uvádí: „*This model costs approximately \$1.66 to run on Replicate... Predictions typically complete within 29 minutes.*” Pro vibe-coding iterace je to nepoužitelné.

`Replicate`

7) Real-world reference (jak to dělají Neural Frames a Kaiber)

- **Neural Frames** = continuous diffusion (Deform-styl): každý frame je SD img2img s předchozím framem jako init + per-frame transform + per-frame prompt blend. Audio reactivity = modulace zoom/strength/prompt blend váhy. Cena: Neural Frames používá kreditový systém podle sekund, ne podle framů — plán **Neural Knight (\$26/měsíc ročně placeno)** poskytuje 2 400 kreditů, Neural Ninja \$66/měsíc dává 7 200 kreditů. Podle Stage and Cinema (prosinec 2025) je sazba „*roughly 15-20 credits per second at 1080p*”; 1 000 framů při 25 FPS = 40 sekund videa = cca 600-800 kreditů ≈ **\$6.50-\$8.65** na Knight tieru. **Pro vibe-codera replikovatelné jen s vlastním backendem** (ne v prohlížeči). Tvrzení Neural Frames, že „*nepoužívá Deform*”, pochází z jejich marketingu — technický mechanismus je z venku k nerozeznání. `OMR + 4`
- **Kaiber** má 3 módy: **Flipbook** (= Deform continuous, single prompt + audio-reactive camera motion, až 8 minut audia), **Motion / Luma Video Flow** (wrapuje Luma Ray, 40 credits / 5 s), **Storyboard / Cuts** (= keyframes + AI image-to-video + střih na beat: „*Cuts coincide with kicks, drops and dynamic shifts. Every transition lands on the music.*”). Storyboard je to, co budeš pravděpodobně chtít napodobit. `OPTIWEB` `KaiberAI`

8) Export videa — WebCodecs je king pro MP4, MediaRecorder jako fallback

- **WebCodecs** `VideoEncoder` + `mp4-muxer` (autor Vanilla, MIT) je v 2026 production-ready cesta. Funguje v Chrome 94+, Edge 94+, Firefox 130+, Safari 26+ (Safari 16.4-18.7 má jen video API, ne `AudioEncoder`). Hardware akcelerace přes Intel Quick Sync / Apple VideoToolbox / NVENC.

- **Workflow:** Pro každý frame canvas → `VideoFrame` → `encoder.encode()` → muxer dostane `EncodedVideoChunk` → finální Blob/file. Audio jde paralelně přes `AudioEncoder` (AAC) nebo zvenku — typicky vezmeš originální Suno MP3 a domuxuješ ho buď přes mp4-muxer (AAC), nebo přes `ffmpeg.wasm`.
- **Offline rendering > real-time capture.** Renderuj frame-by-frame na canvas tak rychle, jak to GPU zvládne (klidně 5× rychleji než real-time), a posílej do encoderu. Real-time `MediaRecorder` (`canvas.captureStream()`) má jitter, je omezený na WebM (VP8/VP9) ve většině prohlížečů a nedává frame-level kontrolu.
- **Doporučené YouTube exportní parametry (Google support „YouTube recommended upload encoding settings“ + zebgardner.com VMAF test 2026):**
 - 1080p30: H.264 High Profile, **8 Mbps** (YouTube oficiální minimum), reálně **10–15 Mbps** pro lepší kvalitu po YouTube re-encode na VP9/AV1
 - 1080p60: 12 Mbps oficiálně, 15–20 Mbps doporučeno (`Longstories`)
 - 4K30: 35–45 Mbps, 4K60: 53–85 Mbps (`Swarmify`)
 - Audio: AAC-LC, 48 kHz, **384 kbps stereo** (YouTube re-encoduje na Opus ~160 kbps) (`Longstories`) (`Magic Hour`)
 - Container: MP4, codec string `avc1.640028` (High Profile Level 4.0) nebo `avc1.42E01E` (Baseline 3.0)
 - **Trik z VMAF testu Zeb Gardnera (2026):** Nahrávej 1440p nebo 4K i s 1080p source — YouTube pak používá VP9 codec při streamování (lepší výsledná kvalita než H.264 fallback pro 1080p uploady). (`TestamentProductions`)

9) PWA specifika a omezení

- `vite-plugin-pwa` je standardní cesta pro React 19 + Vite + TypeScript. Manifest, service worker, ikony, offline cache hotovo s pár řádky configu. (`GitHub`) (`Chapimaster`)
- **Limit na precache:** Workbox má default `maximumFileSizeToCacheInBytes` 2 MiB. Pro AI aplikaci s velkými ONNX modely nebo Three.js bundlem zvyš na `5_000_000` nebo víc, případně **modely nedávej do precache** a načítej je on-demand z IndexedDB / CDN. (`GitHub`)
- **Velké soubory** (Suno track 5–10 MB, exportovaný klip 50–500 MB): použij **File System Access API** (`showSaveFilePicker`) pro stream zápis výsledného videa přímo na disk uživatele bez plnění RAM. Pro audio upload jednoduše `<input type="file">` → `FileReader` / `URL.createObjectURL`.
- **Cross-Origin Isolation:** Pokud používáš `ffmpeg.wasm` (multi-threaded) nebo `SharedArrayBuffer`, musíš nastavit HTTP hlavičky `Cross-Origin-Opener-Policy: same-origin` a `Cross-Origin-Embedder-Policy: require-corp`. Vite-plugin-pwa to neřeší — nakonfiguruj na hostingu (Vercel/Netlify/Cloudflare Pages mají tuhle možnost v nastavení).
- **Mobil:** WebGPU funguje na iOS 26+/Android Chrome 121+, ale **kódování videa přes WebCodecs je výrazně pomalejší a paměťově náročnější**. Pro export plánuj desktop-first.

10) Open-source reference projekty

- **Butterchurn** (jberg/butterchurn) — WebGL2 implementace Milkdrop vizualizeru, MIT, drop-in: 2 řádky kódu pro plnohodnotný audio-reaktivní vizuál s tisíci presety. **Pro MVP nejrychlejší cesta k „je to hudební klip“**. Nevýhoda: vypadá to jako Winamp 2002 (což může být feature i bug). [GitHub](#)
- **Hydra** (hydra-synth/hydra) — live-codable video synth od Olivie Jack, používá Meyda pro FFT analýzu, syntax inspirovaný modular synthesis, čistý WebGL. Hezké pro generativní VJ, ale není to drop-in komponenta. [Medium](#) [GitHub](#)
- **MangoWave** (Louis-Mascari/MangoWave) — komplexní browser-based vizualizér postavený na Butterchurn s il8n, system audio capture, multi-window sync — referenční architektura. [GitHub](#)
- **PinePaper Studio** — komerční browser MP4 exporter, který dokumentuje svůj WebCodecs + mp4-muxer pipeline veřejně (knowledge-hub článek „Browser-Native MP4 Video Export with WebCodecs — How It Works“).
- **Codrops tutoriály 2025** — „Coding a 3D Audio Visualizer with Three.js, GSAP & Web Audio API“, „Interactive Text Destruction with Three.js, WebGPU, and TSL“, „ASTRODITHER“ (audio-reactive TSL experiment od Roberta Borghesi). Všechno produkční vzory.

Details

Doporučená architektura PWA (modulární)

```
React 19 + Vite + TS + Tailwind + vite-plugin-pwa
```

```
UI layer (Tailwind/shadcn)
```

```
├─ Upload audio (Suno MP3/WAV)
```

```
├─ Visual style picker (presety)
```

```
├─ Timeline / section editor (volitelné)
```

```
├─ Preview canvas
```

```
└─ Export dialog (resolution / bitrate)
```

```
Audio pipeline
```

```
AudioContext → MediaElementSource → AnalyserNode
```

```
→ Meyda (BPM, energy, spectral centroid, MFCC)
```

```
→ (volitelně) essentia.js pro song structure
```

```
Visual pipeline (dva módy, switch v UI)
```

```
MÓD A – Generativní (real-time):
```

```
Three.js WebGPURenderer + TSL shadery
```

```
audio uniformy → vertex/fragment shader
```

```
(alt: Butterchurn jako quick-win MVP)
```

```
MÓD B – AI hybrid (offline):  
1. detect sekce v hudbě (intro/verse/chorus/drop)  
2. pro každou sekci → Flux 2 keyframe přes Fal.ai  
3. mezi keyframy → wan-flf2v / pikaframes  
4. concatenate clipy, cuty na beat boundaries  
5. (volitelně) RIFE pro 24→48 FPS smoothing
```

```
Export pipeline  
Render loop: scene.update(audioData[t]) → canvas  
→ VideoFrame → VideoEncoder (avc1.640028, 12 Mbps)  
→ mp4-muxer → Blob  
+ AudioEncoder (AAC 384 kbps) NEBO ffmpeg.wasm mux  
→ File System Access API → uložení MP4 na disk
```

Konkrétní kompromisy

Real-time vs. offline rendering: Pro generativní mód můžeš jet real-time capture (`canvas.captureStream` → `MediaRecorder`), ale dostaneš WebM/VP9, jitter na pomalých strojích a žádnou kontrolu frame timingu. **Vždy preferuj offline rendering:** krmíš encoder konstantním frame-rate (typicky 30 nebo 60 FPS), audio analyzuješ z pre-decoded `AudioBuffer` (ne z `MediaElementSource`), takže můžeš renderovat rychleji než real-time. Pro 3min klip na M2 MacBook Pro je typicky 1-2 min export time.

Web Workers / OffscreenCanvas: Pro plynulé UI běh rendering pipeline ve Web Workeru s `OffscreenCanvas`. Three.js `WebGPURenderer` to podporuje. UI thread zůstane responzivní i během exportu.

FPS pro export:

- 30 FPS — defaultní volba pro YouTube hudební klip, 2× kratší export než 60 FPS
- 60 FPS — má smysl jen pokud máš pohyb (kamera, particles), který z toho profituje; zdvojnásobí bitrate budget

Resoluce:

- 1920×1080 default
- 2560×1440 pokud chceš trigger VP9 codec na YouTube (lepší výsledná kvalita)
- 1080×1920 (9:16) pro Shorts — Pika/Wan to podporují nativně

Kdy NE WebCodecs

WebCodecs neumí `AudioEncoder` na Safari 16.4–18.7 (jen video). Pokud chceš targetovat tyhle starší Safari, použij `MediaRecorder` (WebM) jako fallback + nabídni uživateli „převést na MP4 přes ffmpeg.wasm“ jako jednorázový krok.

Recommendations

Fáze 1 (víkend, MVP, generativní jen):

1. `npm create vite@latest` → React + TS template, přidej Tailwind, vite-plugin-pwa.
2. Implementuj `<audio>` upload + `AudioContext` + `AnalyserNode`. Drop-in **Butterchurn** s default presety. Funkční vizualizér za 2 hodiny.
3. Přidej WebCodecs `VideoEncoder` + `mp4-muxer`. Render canvas → MP4 + audio mux přes `ffmpeg.wasm`. Tohle už je MVP videokliper.
4. Deploy na Vercel/Netlify, nastav COOP/COEP hlavičky.

Fáze 2 (1-2 týdny, polished generativní):

1. Vyhod Butterchurn za **Three.js** `WebGPURenderer` + **TSL shadery** — napiš si vlastní (Claude/Gemini ti TSL kód napíše dobře, je to JS-like).
2. Přidej Meyda pro spectral flux beat detection + 3-5 vizuálních presetů (sphere distortion, particle flow field, fluid sim, kaleidoscope, vector tunnel).
3. Audio-uniformy: kick energy (low band), snare energy (mid), brightness (high band), RMS, beat pulse — namapuj na shader parametry.
4. Offline rendering ve Web Workeru s `OffscreenCanvas` pro non-blocking export.
5. Export presety: 1080p30 12 Mbps, 1080p60 20 Mbps, 1440p30 25 Mbps (YouTube VP9 trigger).

Fáze 3 (volitelná, AI hybrid):

1. Přidej Fal.ai SDK + `.env` s API klíčem (nebo BYO-key UI input). **Nepoužívej Replicate na AnimateDiff** — moc pomalé a drahé pro iteraci (\$1.66, ~29 minut na klip).
2. Implementuj **Storyboard mode**: detekuj sekce písně (chorus/verse/drop), generuj 1 keyframe Flux 2 pro každou, pak `wan-1.1f2v` nebo `pikaframes` mezi sousedními. Cuty timeline na beat boundaries.
3. Compositing: AI klip pod-vrstva + algoritmický overlay (particles, light leaks) přes blend mode v shaderu. Tohle ti dá AI estetiku s audio-reaktivitou.
4. Cache AI klipy v IndexedDB — uživatel většinou iteruje na timeline, ne na keyframech.

Co NEdělej:

- Negeneruj diffusion video v prohlížeči — počkej 1-2 roky.
- Nepoužívej `MediaRecorder` pro finální export pokud máš WebCodecs (WebM kvalita, jitter, žádná frame kontrola).
- Nedávej velké modely (>10 MB) do service worker precache — načítej on-demand a cachuj v IndexedDB sám.
- Nepředpokládej mobilní GPU paritu — testuj export jen na desktopu, na mobilu nabídní jen preview.

Benchmarky pro change-of-strategy:

- Pokud WebGPU na cílových mobilech > 95 % → můžeš odstranit WebGL2 fallback.
- Pokud Fal.ai Pikaframes klesne pod \$0.02/sec → AI hybrid se stane default módem.
- Pokud SVD nebo Wan dostane WebGPU port → můžeš mít plně offline AI generování.
- Pokud Suno přidá oficiální video API → integruj přímo, vyhni se mezi-uploadu.

Caveats

- **Fal.ai ceny** uvedené výše jsou z jejich live model stránek (květen 2026); mohou se měnit. Replicate ceny jsou „approximately“ jejich vlastní UI (skutečný čas predikce variuje).
- **WebGPU mobilní podpora** je stále křehká — Firefox Android za flagem, Android Chrome jen na zařízeních s Android 12+/Qualcomm/ARM/Imagination GPU. Plánuj UX pro WebGL2-only mobilní uživatele. ([web.dev](#))
- **VMAF/PSNR test od Zeb Gardnera** (Best Upload Settings 2026) je single-channel test; tip „nahrávej 1440p místo 1080p pro lepší VP9 codec“ je široce reportovaný v komunitě, ale YouTube to ve své dokumentaci explicitně neuvádí.
- **AI hudba a licence** se v posledních měsících rychle mění. Suno Pro/Premier stále poskytuje commercial rights k 5/2026, ale situace u Udio (settlement s UMG 29. října 2025, s Warner Music Group 19. listopadu 2025) ukazuje, že to může kdykoli skončit.
- **Neural Frames „nepoužívá Deforum“** je tvrzení z jejich vlastních marketing materiálů recyklované několika SEO aggregátory; technický mechanismus (continuous SD img2img s prompt scheduling + audio modulation) je z venku k nerozeznání od Deforum-stylu. Subagentní výzkum nenašel nezávislé potvrzení.
- **WebCodecs Firefox podpora** byla doplněna v Firefox 130 (2024), ale ([AudioEncoder](#)) je v některých verzích nestabilní — testuj na konkrétních cílových verzích.
- ([ffmpeg.wasm](#)) **má praktický 2 GB memory limit** v prohlížeči — pro 4K klipy delší než ~3 minuty narazíš. Hledej alternativy (WebCodecs-only pipeline bez ffmpeg.wasm) nebo nabídní server-side mux jako paid feature.
- **Stable Diffusion Turbo benchmark 1 s na RTX 4090** je z Microsoft Open Source blogu z února 2024 — na běžných uživatelských GPU počítej s 5–30× pomalejší inferencí; pro consumer mobilní GPU prakticky nepoužitelné na real-time klip.